Problem 4: Give an algorithm that take in two strings A and B (of lengths n and m) and find the longest suffix of A that exactly matches the prefix of B. This algorithm should run on O(n + m) time. ?

Solution: The problem of finding the longest suffix of A that matches with the suffix of B is very relevant as a sub-problem of shortest common superstring problem. O(m+n) run time could be achieved by using a suffix tree.

There would n suffixes of A. Create a suffix tree of A. While creating the suffix tree, we append $ to every suffix of A, where $ is a character that is not present in A or B. $ denotes end of the string. Creating suffix tree of a string could be done in linear time and hence this operation is O(n).

Now search for the string B in this suffix tree. The search will end at some node in the suffix tree where it is no longer able to match more characters from B (because no branch exists with a match or because it is done with matching all of B).

We need to ensure that this sequence corresponds to a suffix of A and not just the prefix of a suffix of A. To take care of this we will check whether this node has a $ branch. If not, we search for the lowest common ancestor of this node that has a $ branch. String obtained by traversing from root to this node would correspond to the longest suffix of A that is a prefix of B. This step for searching is linear in length of B and hence the algorithm is O(n+m)

Problem 5

Explain how to adapt the algorithm for computing the longest palindrome within a string to the notion of biological palindromes", where we seek the longest substring of a DNA sequence which is its own reverse complement.

Answer:

Let S be the interesting DNA sequence, and $\bar{S}$ be its reverse complement. Build suffix tree for sequence $C = S\#\bar{S}$. Then for all $i \in [0, n)$, find the LCA of the two suffixes of C with length i and $2n-i+1$. Suppose the depth (represented by d) of the LCA is the largest when $i = c$, then the longest biological palindrome is centered at c. The complexity of building suffix for C is $O(n)$ (where n is length of S, and the complexity of LCA is constant (executed n times), therefore the complexity for the whole algorithm is also $O(n)$.

Problem 6

Consider the problem of partitioning a string into a sequence of palindromes (pieces that read the same forward as backwards). Since each single character is a palindrome, this can always be done — but we seek the partitioning into the minimum number of pieces possible. Thus "AMMADAM" can be partitioned into three pieces, "A-M-MADAM" and "ABBABABA" can be partitioned into three pieces as "ABBA-B-ABA"

(a) Does the greedy heuristic (find the largest palendrome) always work? Give a proof or a counterexample?

5

Answer:

Not always work. Counterexample: ACAAAA

(b) Does repeatedly finding the largest left-most remaining palendrome always work? Give a proof or a counterexample?

5

Answer:

Not always work. Counterexample: AAAACA

(c) Give an efficient algorithm for finding the optimal partitioning.

Answer: Let $f(n)$ be the best partition for substring $A[1:n]$, and $P(n) = \{i, A[i:n] \text{ is a palindrome}\}$. By dynamic programming, we have

$$f(n+1) = \min_{i \in P(n+1)} (f(i-1)+1)$$

15

There are at most $O(n^2)$ palindromes and in the above algorithm every palindrome is examined once, so it runs in $O(n^2)$ time. As long as we can obtain all palindromes within the string in $O(n^2)$ time, we can conclude that the whole algorithm find the shortest palindrome partition of a string in $O(n^2)$ time.

In the longest palindrome algorithm using suffix tree, the longest palindrome starts at each position i will be examined. Similarily, we can find all palindromes by finding the lowest common ancestor (LCAs) of $S[i]$ and $S[2n+1-i]$ for each $1 \leq i \leq n$ and take the pathes from the LCAs to the root. Every node in the pathes corresponds to a palindrome. Therefore, we can work out all palindromes in $O(n^2)$.

In conclusion, the order of complexity of the whole algorithm is $O(n^2)$.

Moreover, we do not have to goes through all elements in $P(n+1)$ during dynamic programming. For example, we can use the greedy heuristic to work out a result as a start point. During the dynamic programming, we can use it to prune and update the current best result if necessary.